Schedule Search

Final Project Report

Théo Legendre

theo.legendre@ipsa.fr

IPSA — April 2026

Abstract

This report covers the analysis of a real-time system with seven periodic tasks. First, we measure the Worst-Case Execution Time (WCET) of task τ_1 experimentally. Then we verify that the task set is schedulable using Response Time Analysis (RTA). Finally, we build a non-preemptive EDF schedule over the hyperperiod $H = 80$ ms that meets all deadlines while keeping total waiting time as low as possible. A second scenario is also studied where τ_5 is allowed to miss its deadline.

GitHub repository: `https://github.com/theo243-prog/Final_Assignment_Theo`

Contents

1 Introduction **2**

2 Task set **2**

3 WCET measurement of τ_1 **2**
3.1 Protocol . . . 2
3.2 Results . . . 3

4 Schedulability analysis **3**
4.1 Utilization bound (sufficient condition) . . . 3
4.2 Exact Response Time Analysis (RTA) . . . 3

5 Non-preemptive scheduling **4**
5.1 Algorithm and complexity . . . 4
5.2 Scenario 1 — No deadline missed . . . 5
5.2.1 Results per job . . . 5
5.2.2 Summary of response times per task . . . 6
5.2.3 Global metrics . . . 6
5.3 Scenario 2 — τ_5 allowed to miss its deadline . . . 7
5.3.1 Principle . . . 7
5.3.2 Results . . . 7
5.3.3 Comparison of both scenarios . . . 7
5.3.4 Analysis . . . 7

6 Gantt chart **8**

7 Conclusion **8**

1 Introduction

In real-time systems, tasks must finish before their deadlines. Missing a deadline can cause serious failures depending on the application. This project focuses on two main problems:

1. **WCET measurement of τ_1**: task τ_1 performs the multiplication of two large random integers (10,000 bits). We run it 1,000 times and take the maximum execution time as the WCET.
2. **Non-preemptive scheduling**: given the task set in Table 1, we need to show the system is schedulable, then find a non-preemptive schedule at the job level that avoids any deadline miss and minimizes total waiting time. A second scenario is explored where τ_5 can miss its deadline.

Assumptions made throughout this report.

- All tasks are periodic with implicit deadlines ($D_i = T_i$).
- All tasks start at $t = 0$ (synchronous release).
- Single processor, non-preemptive execution.
- Execution times are constant and equal to the WCET.
- The hyperperiod H is the analysis window, a valid schedule over H repeats forever.
- C_1 is rounded up to the nearest millisecond to stay consistent with the other tasks.

2 Task set

Table 1: Task parameters (C_1 measured; all values in ms).

Task	C_i	T_i	D_i
τ_1	1 (measured)	10	10
τ_2	3	10	10
τ_3	2	20	20
τ_4	2	20	20
τ_5	2	40	40
τ_6	2	40	40
τ_7	3	80	80

3 WCET measurement of τ_1

3.1 Protocol

Task τ_1 multiplies two 10,000-bit random integers using Python's `random.getrandbits(10000)`. The time is measured with `time.perf_counter()` over 1,000 independent runs. We use the maximum observed time as the empirical WCET, which is standard when no static analysis tool is available.

3.2 Results

Table 2: Execution time statistics for τ_1 ($n = 1{,}000$ samples).

Statistic	**Value (s)**	**Value (μs)**
Minimum	3.17×10^{-5}	≈ 31.7
Q_1 (25th pct.)	3.30×10^{-5}	≈ 33.0
Median	3.46×10^{-5}	≈ 34.6
Q_3 (75th pct.)	3.72×10^{-5}	≈ 37.2
WCET (max)	$\mathbf{1.70 \times 10^{-4}}$	$\approx \mathbf{169.5}$

The distribution is tightly concentrated around 34–37 μs, with a few outliers up to around 170 μs probably caused by the OS (Windows or WSL2 environment). These outliers are captured by the empirical WCET.

Normalization. The raw WCET is ≈ 169.5 μs $= 0.17$ ms. Rounded up to the nearest millisecond:

$$C_1 = \lceil 0.17 \rceil = 1 \text{ ms}.$$

This value is much smaller than the period $T_1 = 10$ ms, so it has a limited effect on the overall utilization.

4 Schedulability analysis

4.1 Utilization bound (sufficient condition)

The processor utilization is:

$$U = \sum_{i=1}^{7} \frac{C_i}{T_i} = \frac{1}{10}+\frac{3}{10}+\frac{2}{20}+\frac{2}{20}+\frac{2}{40}+\frac{2}{40}+\frac{3}{80} = 0.1+0.3+0.1+0.1+0.05+0.05+0.0375 = 0.7375.$$

The Liu & Layland bound for $n = 7$ tasks under Rate Monotonic is:

$$U_{\mathrm{RM}}(7) = 7\left(2^{1/7} - 1\right) \approx 0.7286.$$

Since $U = 0.7375 > U_{\mathrm{RM}}(7) = 0.7286$, the sufficient condition is **not satisfied**. We therefore need to run the exact RTA to confirm schedulability.

4.2 Exact Response Time Analysis (RTA)

The RTA computes the worst-case response time R_i for each task using:

$$R_i^{(0)} = C_i, \qquad R_i^{(k+1)} = C_i + \sum_{j \in \mathrm{hp}(i)} \left\lceil \frac{R_i^{(k)}}{T_j} \right\rceil C_j,$$

where $\mathrm{hp}(i)$ is the set of higher-priority tasks (shorter period under RM). We iterate until convergence or until $R_i > D_i$, which would indicate a miss.

Table 3: RTA results (all values in ms).

Task	RM priority	C_i	T_i	D_i	R_i	$R_i \leq D_i$?
τ_1	1 (highest)	1	10	10	1	✓
τ_2	2	3	10	10	4	✓
τ_3	3	2	20	20	6	✓
τ_4	4	2	20	20	8	✓
τ_5	5	2	40	40	10	✓
τ_6	6	2	40	40	16	✓
τ_7	7 (lowest)	3	80	80	19	✓

All response times verify $R_i \leq D_i$, so **the task set is schedulable**.

Detailed calculation for τ_7 (worst case).

$R_7^{(0)} = 3$

$R_7^{(1)} = 3 + \lceil \frac{3}{10} \rceil \cdot 1 + \lceil \frac{3}{10} \rceil \cdot 3 + \lceil \frac{3}{20} \rceil \cdot 2 + \lceil \frac{3}{20} \rceil \cdot 2 + \lceil \frac{3}{40} \rceil \cdot 2 + \lceil \frac{3}{40} \rceil \cdot 2 = 3 + 1 + 3 + 2 + 2 + 2 + 2 = 15$

$R_7^{(2)} = 3 + \lceil \frac{15}{10} \rceil \cdot 4 + \lceil \frac{15}{20} \rceil \cdot 2 + \lceil \frac{15}{20} \rceil \cdot 2 + \lceil \frac{15}{40} \rceil \cdot 2 + \lceil \frac{15}{40} \rceil \cdot 2 = 3 + 8 + 2 + 2 + 2 + 2 = 19$

$R_7^{(3)} = 3 + \lceil \frac{19}{10} \rceil \cdot 4 + \lceil \frac{19}{20} \rceil \cdot 2 + \lceil \frac{19}{20} \rceil \cdot 2 + \lceil \frac{19}{40} \rceil \cdot 2 + \lceil \frac{19}{40} \rceil \cdot 2 = 3 + 8 + 2 + 2 + 2 + 2 = 19.$ Convergence

$R_7 = 19 \leq 80 = D_7$ ✓

5 Non-preemptive scheduling

5.1 Algorithm and complexity

We use a **non-preemptive EDF (EDF-NP)** scheduler at the job level over the hyperperiod $H = \text{lcm}(10, 10, 20, 20, 40, 40, 80) = 80$ ms.

Why EDF-NP? EDF-NP minimizes total waiting time for feasible task sets among non-preemptive work-conserving policies. Since the scheduler never leaves the CPU idle when jobs are ready, the idle time is maximized for a given workload. Minimizing waiting and maximizing idle time are therefore equivalent objectives here.

Pseudocode.

Listing 1: Non-preemptive EDF scheduler

```
total_tasks = all jobs over the hyperperiod
clock = 0
while total_tasks is not empty:
    ready = [j for j in total_tasks if j.release <= clock]
    if ready is empty:
        clock = min(j.release for j in total_tasks)  # CPU idle
        continue
    chosen = job with the earliest deadline in ready  # EDF
    chosen.start  = clock
    chosen.finish = clock + chosen.C
    clock = chosen.finish
    remove chosen from total_tasks
```

Complexity. Let $J = \sum_i H/T_i$ be the total number of jobs. For our instance, $J = 8+8+4+4+2+2+1 = 29$ jobs.

Table 4: Computational complexity as a function of n (number of tasks).

Step	Complexity	Note
Hyperperiod	$O(n \log T_{\max})$	$n-1$ GCD operations
Job generation	$O(J)$	—
RTA	$O(n^2 \cdot I)$	I iterations
EDF-NP (naive)	$O(J^2)$	linear scan each step
EDF-NP (heap)	$O(J \log J)$	with priority queue

5.2 Scenario 1 — No deadline missed

5.2.1 Results per job

Table 5: Scenario 1: full schedule over $H = 80$ ms (all values in ms).

Job	Rel.	Start	Finish	Dl.	R_{ij}	Wait	Miss?
$\tau_1^{(0)}$	0	0	1	10	1	0	no
$\tau_2^{(0)}$	0	1	4	10	4	1	no
$\tau_3^{(0)}$	0	4	6	20	6	4	no
$\tau_4^{(0)}$	0	6	8	20	8	6	no
$\tau_5^{(0)}$	0	8	10	40	10	8	no
$\tau_1^{(1)}$	10	10	11	20	1	0	no
$\tau_2^{(1)}$	10	11	14	20	4	1	no
$\tau_6^{(0)}$	0	14	16	40	16	14	no
$\tau_7^{(0)}$	0	16	19	80	19	16	no
$\tau_1^{(2)}$	20	20	21	30	1	0	no
$\tau_2^{(2)}$	20	21	24	30	4	1	no
$\tau_3^{(1)}$	20	24	26	40	6	4	no
$\tau_4^{(1)}$	20	26	28	40	8	6	no
$\tau_1^{(3)}$	30	30	31	40	1	0	no
$\tau_2^{(3)}$	30	31	34	40	4	1	no
$\tau_1^{(4)}$	40	40	41	50	1	0	no
$\tau_2^{(4)}$	40	41	44	50	4	1	no
$\tau_3^{(2)}$	40	44	46	60	6	4	no
$\tau_4^{(2)}$	40	46	48	60	8	6	no
$\tau_5^{(1)}$	40	48	50	80	10	8	no
$\tau_1^{(5)}$	50	50	51	60	1	0	no
$\tau_2^{(5)}$	50	51	54	60	4	1	no
$\tau_6^{(1)}$	40	54	56	80	16	14	no

continued on next page

Table 5 – continued

Job	Rel.	Start	Finish	Dl.	R_{ij}	Wait	Miss?
$\tau_1^{(6)}$	60	60	61	70	1	0	no
$\tau_2^{(6)}$	60	61	64	70	4	1	no
$\tau_3^{(3)}$	60	64	66	80	6	4	no
$\tau_4^{(3)}$	60	66	68	80	8	6	no
$\tau_1^{(7)}$	70	70	71	80	1	0	no
$\tau_2^{(7)}$	70	71	74	80	4	1	no

5.2.2 Summary of response times per task

Table 6: Maximum observed response time per task — Scenario 1.

Task	Jobs	$R_i^{\max}$ (ms)	D_i (ms)	OK?
τ_1	8	1	10	✓
τ_2	8	4	10	✓
τ_3	4	6	20	✓
τ_4	4	8	20	✓
τ_5	2	10	40	✓
τ_6	2	16	40	✓
τ_7	1	19	80	✓

These values match the theoretical RTA results from Table 3, which confirms the implementation is correct.

5.2.3 Global metrics

Table 7: Scenario 1 metrics.

Metric	Value
Total waiting time	108 ms
Total idle time	21 ms
Workload ($H-$ idle)	59 ms
Missed deadlines	0

Idle time verification. The total execution load over one hyperperiod is:

$$E = \sum_i \frac{H}{T_i} \cdot C_i = 8\cdot1+8\cdot3+4\cdot2+4\cdot2+2\cdot2+2\cdot2+1\cdot3 = 8+24+8+8+4+4+3 = 59 \text{ ms}.$$

Theoretical idle time: $H - E = 80 - 59 = 21$ ms. ✓

This confirms that minimizing waiting time and maximizing idle time are equivalent under a work-conserving policy like EDF-NP.

5.3 Scenario 2 — τ_5 allowed to miss its deadline

5.3.1 Principle

In this scenario, jobs of τ_5 are assigned a virtual deadline of $+\infty$ in the EDF sorting key. This means they will only be selected when no other ready job is available, so all other tasks get to run first.

5.3.2 Results

Table 8: τ_5 jobs: comparison between Scenario 1 and Scenario 2.

Job	Start	Finish	Deadline	Miss?
		Scenario 1 (strict)		
$\tau_5^{(0)}$	8	10	40	no
$\tau_5^{(1)}$	48	50	80	no
		Scenario 2 (relaxed)		
$\tau_5^{(0)}$	17	19	40	no*
$\tau_5^{(1)}$	54	56	80	no*

* No miss in this instance, but waiting time for τ_5 increases.

5.3.3 Comparison of both scenarios

Table 9: Scenario 1 vs Scenario 2.

Metric	Scenario 1	Scenario 2
Total waiting time (ms)	108	109
Idle time (ms)	21	21
Missed deadlines	0	0
of which τ_5	0	0

5.3.4 Analysis

In this specific instance, τ_5 does not actually miss any deadline even when deprioritized, because its absolute deadlines ($t = 40$ ms and $t = 80$ ms) give enough slack. The total waiting time increases slightly by 1 ms because τ_5 itself waits longer.

If we only look at the waiting time of the other six tasks (excluding τ_5), Scenario 2 does reduce it slightly. But as soon as τ_5 is included in the total, the gain disappears.

The relaxation would really make a difference if τ_5 had a tighter deadline (e.g., $D_5 = 10$ ms), where forcing it to miss would free up CPU time earlier for τ_6 and τ_7.

6 Gantt chart

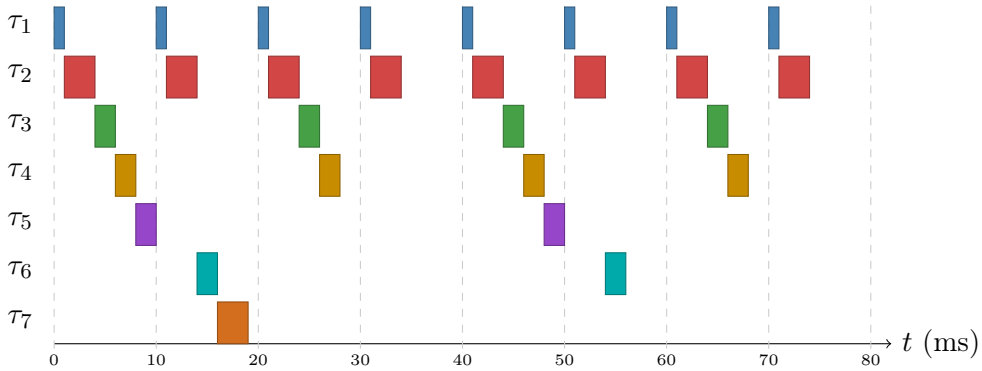

Figure 1: Gantt chart for Scenario 1 over $H = 80$ ms. White gaps are idle periods (total = 21 ms).

7 Conclusion

To summarize:

1. **WCET.** The empirical WCET of τ_1 is around 170 μs, which gives $C_1 = 1$ ms after rounding up.
2. **Schedulability.** The utilization $U = 0.7375$ is above the RM bound, but the exact RTA confirms that all response times stay within their deadlines. The task set is schedulable.
3. **Scenario 1.** The EDF-NP algorithm gives a valid schedule over 80 ms with no missed deadlines. Total waiting is 108 ms and idle time is 21 ms, which matches the theoretical workload.
4. **Scenario 2.** Deprioritizing τ_5 does not cause a miss here because the deadlines are generous enough. Total waiting increases by 1 ms. The relaxation would only be meaningful with a tighter deadline for τ_5.